

Optimization

6.5 Direct MultiSearch (DMS)

José Firmino Aguilar Madeira

6.4 ended with the question: can we build an approximation of the front in a single run?

Direct search (3.2.5; 5.1): one iterate

$$x_k$$

DMS (Direct MultiSearch): a list of nondominated points

$$L_k = \{x^1, \dots, x^q\}$$

6.4 ended with the question: can we build an approximation of the front in a single run?

Direct search (3.2.5; 5.1): one iterate

$$x_k$$

DMS (Direct MultiSearch): a list of nondominated points

$$L_k = \{x^1, \dots, x^q\}$$

How do we do direct search when there is no single “best” point?

That is the question DMS answers.

From direct search to DMS

Direct search (one objective)

1. choose x_k ;
2. poll $x_k + \alpha_k d$, $d \in D_k$;
3. a point gets in if it has a **smaller f** .

Success = finding a point with smaller f .

DMS (several objectives)

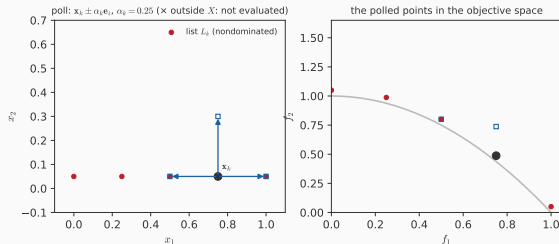
1. choose a centre x_k **from the list**;
2. poll $x_k + \alpha_k d$, $d \in D_k$;
3. a point gets in if it **survives the dominance filter**.

Success = **the list changed**, $L_{k+1} \neq L_k$.

Nothing is aggregated: there are no weights and no bounds — only dominance (6.2).

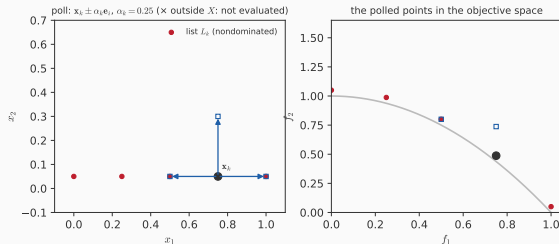
With $m = 1$, DMS is exactly directional direct search.

One poll step: who enters the list?



The course's nonconvex example, $\alpha_k = 0.25$. The point outside X is not even evaluated. What do we do with each new point?

One poll step: who enters the list?



The course's nonconvex example, $\alpha_k = 0.25$. The point outside X is not even evaluated. What do we do with each new point?

dominated



discard

nondominated



enters the list

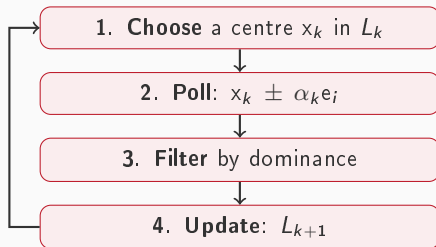
dominates points of the list



enters, and those leave

That is practically the whole mechanism of DMS.

One iteration in four steps



Success

$L_{k+1} \neq L_k$: the list changed. The step size is kept (or increased).

Failure

$L_{k+1} = L_k$: the list did not change. α_k decreases — only that of the polled centre.

The list: $L_k = \{(x, \alpha)\}$

Each element of the list stores

- a feasible **solution** x ;
- its **step size** α — each point has its own.

The points of the list are **mutually nondominated**:
at each iteration, the list is the **current
nondominated approximation**.

The centre is chosen from the list and stays in it as long as it is not dominated.

Why one step size per point?

In a region already well explored, the step size of that point has already decreased; a new point, in another part of the front, starts with a larger step size. The list refines some parts without stopping the exploration of others.

- filter: add the polled points to the list and keep only the nondominated ones.
- select: decide how much of the filtered list to keep. In this course, all of it: $L_{\text{trial}} = L_{\text{filt}}$, because we want to approximate the whole front.
- order: choose who is polled next. Simple rule (FIFO): the polled centre goes to the end of the list — all end up being polled.

Other options (paper)

- order by the largest gap Γ between neighbours on the front;
- start with several points ($\text{DMS}(n, \text{line})$);
- cache: do not re-evaluate points already evaluated.

The DMS algorithm (Algorithm 2.1 of the paper)

DMS — given $x_0 \in X$, $\alpha_0 > 0$, $0 < \beta_1 \leq \beta_2 < 1$,
 $\gamma \geq 1$, $\mathcal{D}$; $L_0 = \{(x_0, \alpha_0)\}$

```
1: for  $k = 0, 1, 2, \dots$  do
2:   Selection: order  $L_k$ ;  $(x_k, \alpha_k) \leftarrow$  first element
3:   Search (optional): evaluate  $f_X$  on a finite set; if successful,
    skip 4–5
4:   Poll: evaluate  $f_X$  on  $\{x_k + \alpha_k d : d \in D_k\}$ ;  $L_{\text{add}} \leftarrow \{(x_k + \alpha_k d, \alpha_k)\}$ 
5:    $L_{\text{filt}} \leftarrow \text{filter}(L_k, L_{\text{add}})$ ;  $L_{\text{trial}} \leftarrow \text{select}(L_{\text{filt}})$ 
6:   if  $L_{\text{trial}} \neq L_k$ : success,  $L_{k+1} \leftarrow L_{\text{trial}}$ ; else failure,  $L_{k+1} \leftarrow L_k$ 
7:   Step size: success  $\Rightarrow \alpha_{\text{new}} \in [\alpha_k, \gamma \alpha_k]$  at the new points and
    at  $x_k$ , if it stays in the list; failure  $\Rightarrow \alpha_{\text{new}} \in [\beta_1 \alpha_k, \beta_2 \alpha_k]$  at  $x_k$ 
8: end for
```

Essential

selection + poll + filter

Search step: optional (spreading, models). **The poll step supports the theory** — as in GLODS (5.1).

Cost: up to $|D_k|$ evaluations of the vector f per poll step ($2n$ with $[I - I]$); parallelizable. Stopping: $\alpha < \alpha_{\min}$ on the whole list, or budget $N_{\max}$.

SP1: one iteration by hand

$$\min ((x_1 - 1)^2 + (x_1 - x_2)^2, (x_1 - x_2)^2 + (x_2 - 3)^2), -1 \leq x_1, x_2 \leq 5; \quad L_0 = \{(1.5, 1.5)\}, \quad \alpha_0 = 1, \\ D = [I \quad -I].$$

Centre (1.5, 1.5): $f = (0.25, 2.25)$.

Polled $x_k \pm e_j$:

point	f
(2.5, 1.5)	(3.25, 3.25)
(1.5, 2.5)	(1.25, 1.25)
(0.5, 1.5)	(1.25, 3.25)
(1.5, 0.5)	(1.25, 7.25)

Iterations 1 and 2 by hand are in the paper (Section 3). The original MATLAB code of DMS (www.mat.uc.pt/dms) is distributed in class.

SP1: one iteration by hand

$$\min ((x_1 - 1)^2 + (x_1 - x_2)^2, (x_1 - x_2)^2 + (x_2 - 3)^2), -1 \leq x_1, x_2 \leq 5; \quad L_0 = \{(1.5, 1.5)\}, \quad \alpha_0 = 1, \\ D = [I \quad -I].$$

Centre (1.5, 1.5): $f = (0.25, 2.25)$.

Polled $x_k \pm e_j$:

point	f
(2.5, 1.5)	(3.25, 3.25)
(1.5, 2.5)	(1.25, 1.25)
(0.5, 1.5)	(1.25, 3.25)
(1.5, 0.5)	(1.25, 7.25)

Filter. (1.25, 1.25) and (0.25, 2.25) are incomparable: **it enters**. The other three are dominated by (1.25, 1.25): **discarded**.

Iterations 1 and 2 by hand are in the paper (Section 3). The original MATLAB code of DMS (www.mat.uc.pt/dms) is distributed in class.

SP1: one iteration by hand

$\min ((x_1 - 1)^2 + (x_1 - x_2)^2, (x_1 - x_2)^2 + (x_2 - 3)^2), -1 \leq x_1, x_2 \leq 5; L_0 = \{(1.5, 1.5)\}, \alpha_0 = 1,$
 $D = [I \ -I].$

Centre (1.5, 1.5): $f = (0.25, 2.25).$

Polled $x_k \pm e_j$:

point	f
(2.5, 1.5)	(3.25, 3.25)
(1.5, 2.5)	(1.25, 1.25)
(0.5, 1.5)	(1.25, 3.25)
(1.5, 0.5)	(1.25, 7.25)

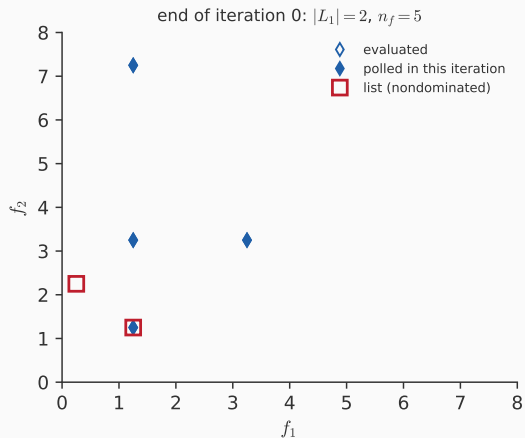
Filter. (1.25, 1.25) and (0.25, 2.25) are incomparable: **it enters**. The other three are dominated by (1.25, 1.25): **discarded**.

Result. $L_1 = \{(1.5, 2.5), (1.5, 1.5)\}$: the list went from **1 to 2** points. **Success**; $\alpha = 1$ is kept. $n_f = 1 + 4 = 5$.

Iterations 1 and 2 by hand are in the paper (Section 3). The original MATLAB code of DMS (www.mat.uc.pt/dms) is distributed in class.

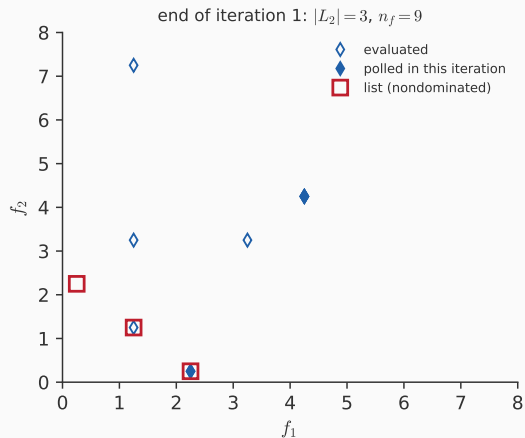
SP1: the front emerges

End of iteration 0: $|L_1| = 2$ (previous page).



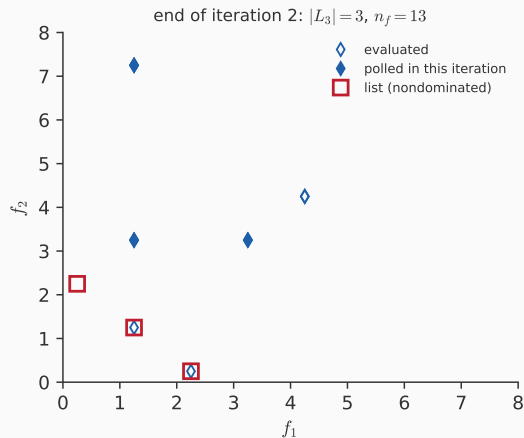
SP1: the front emerges

End of iteration 1: one more point enters, $|L_2| = 3$.



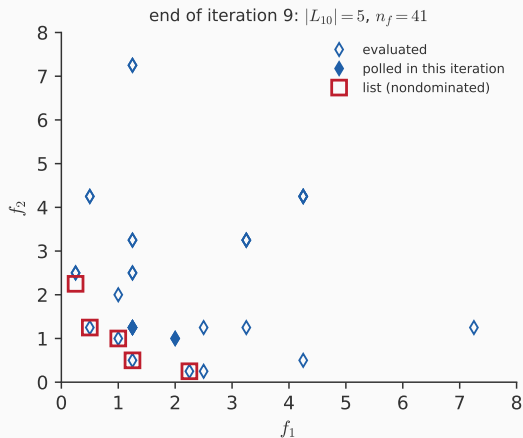
SP1: the front emerges

End of iteration 2: the list does not change — **failure**: $|L_3| = 3$ and the step size of the centre becomes $\frac{1}{2}$.



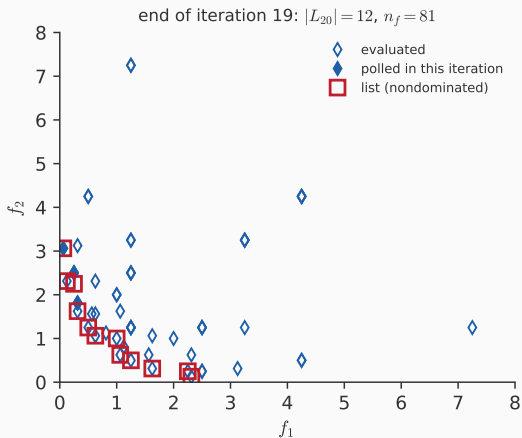
SP1: the front emerges

After 10 iterations: $|L_{10}| = 5$,
 $n_f = 41$.



SP1: the front emerges

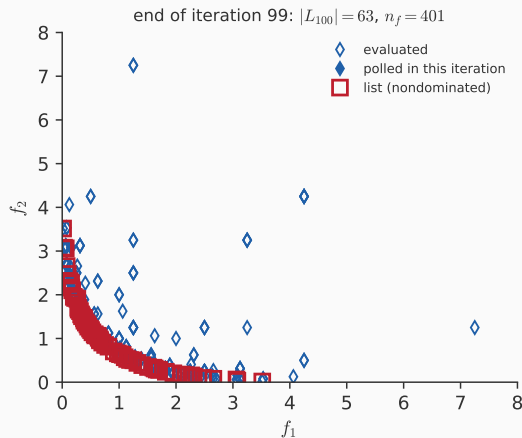
After 20: $|L_{20}| = 12$, $n_f = 81$.



SP1: the front emerges

After 100: $|L_{100}| = 63$, $n_f = 401$.

An approximation of the front
emerges from direct search itself
— with no weights and no
bounds.



Success: $L_{k+1} \neq L_k$

A point does not need to dominate the centre: it is enough to **improve the nondominated approximation** (at iteration 0 of SP1, the new point did not dominate the centre — it was incomparable).

Failure: $L_{k+1} = L_k$

No polled point changes the list. Then α decreases **only for the polled centre** (SP1, iteration 2: $\alpha = 1 \rightarrow \frac{1}{2}$ only at (1.5, 1.5)).

Constraints: extreme barrier (5.1)

$$f_X(x) = \begin{cases} f(x), & x \in X \\ (+\infty, \dots, +\infty), & x \notin X \end{cases}$$

Outside X **the objectives are not even evaluated**: it is enough to know “feasible or not”. It works for black-box constraints.

What does stationarity mean with several objectives?

One objective: there is no feasible direction along which f decreases.

Several objectives: there is no feasible direction along which **all the objectives decrease at the same time**.

Pareto–Clarke critical point

$$\forall d \in T_X^{Cl}(x^*) \quad \exists i : f_i^\circ(x^*; d) \geq 0.$$

If the f_i are strictly differentiable (e.g. C^1):

$$\forall d \in T_X^{Cl}(x^*) \quad \exists i : \nabla f_i(x^*)^T d \geq 0.$$

T_X^{Cl} : Clarke tangent cone; f_i° : generalized directional derivative (Clarke–Jahn), for f_i only L -Lipschitz (R).

Definition in 5.1.

What does stationarity mean with several objectives?

One objective: there is no feasible direction along which f decreases.

Several objectives: there is no feasible direction along which **all the objectives decrease at the same time**.

It does **not** mean $\nabla f_i = 0$ for all i : it is enough that, in each direction, *some* objective does not decrease.

Pareto–Clarke critical point

$$\forall d \in T_X^{Cl}(x^*) \quad \exists i : f_i^\circ(x^*; d) \geq 0.$$

If the f_i are strictly differentiable (e.g. C^1):

$$\forall d \in T_X^{Cl}(x^*) \quad \exists i : \nabla f_i(x^*)^T d \geq 0.$$

T_X^{Cl} : Clarke tangent cone; f_i° : generalized directional derivative (Clarke–Jahn), for f_i only L -Lipschitz (R).

Definition in 5.1.

What DMS guarantees — and what it does not

With iterates in a compact set (here, X bounded) and a rational mesh or sufficient decrease $\Rightarrow \alpha_k \rightarrow 0$
along a subsequence of unsuccessful iterations $\Rightarrow$ limit point x_*

$\Rightarrow f_i$ Lipschitz near x_* and dense refining directions: x_* is Pareto–Clarke critical

With fixed D (as here, $D = [I \ -I]$): for each $d \in D$, $\exists i : f_i^\circ(x_*; d) \geq 0$ (only in those directions).

It guarantees

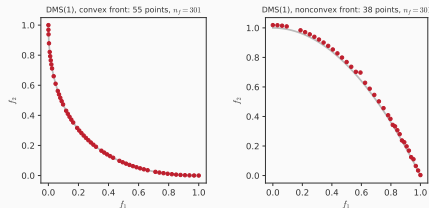
- a limit point with the appropriate stationarity;
- without derivatives (only local Lipschitz), with constraints by extreme barrier;
- with $m = 1$, the theory of direct search.

It does not guarantee

- that *the whole list* converges to the front;
- that this point is efficient (not even locally);
- a good distribution along the front.

“Global convergence”: no need to start close to the solution — it does **not** mean global optimum. A **local necessary** condition.

DMS on the course's two examples



DMS(1) from the centre of the box, $\alpha_0 = 1$, $D = [I \ -I]$, FIFO, no cache, $N_{\max} = 300$ (the ongoing poll step is completed: $n_f \leq N_{\max} + 2n - 1$, counting the initial evaluation). Convex: **55 points**, HV 1.641 (exact 1.648); nonconvex: **38 points**, HV 1.116 (exact 1.148), with $r = (1.1, 1.65)$.

One run builds a whole approximation. In 6.3/6.4, each point was a full optimization. (One run with hundreds of evaluations: it is not automatically cheaper. A proper comparison: HV vs. n_f , 6.6.)

The implementation matters

Variants, on the nonconvex example (our results),
 $N_{\max} = 300$:

variant	it.	n_f	$ L $	HV
DMS(1), FIFO	98	301	38	1.116
DMS(n ,line), FIFO	108	302	50	1.136
DMS(1), spread Γ	97	301	40	1.118
DMS(1), FIFO, cache	232	303	79	1.138

Do not re-evaluate what we already know

Without a cache, **more than half** of the evaluations repeated points already evaluated (about one third, points of the list itself). With it, the same budget gives more iterations and twice as many points.

Starting with several points (a segment) also helps; the spread ordering changes little *here*.

The 2011 paper and a real application

Tests: 100 bound-constrained problems, $1 \leq n \leq 30$, $2 \leq m \leq 4$, common budget of 20 000 evaluations; comparison with AMOSA, BIMADS and NSGA-II.

- DMS was **competitive** in the quality of the fronts;
- very good **spread**;
- NSGA-II was slightly more efficient in one of the distribution metrics.

On this set of problems, with these parameters and this budget. The details (metrics, profiles) are in the paper.

OPTIMIZATION IN PRACTICE

Viscoelastic sandwich panels: maximize the damping, minimize mass and cost; finite element model. **The DMS used works only with real variables:** the discrete choices (material and thickness of each layer, orientation, core thickness) were *encoded* in 25 real variables in $[0, 4]$. With mass ≤ 1.5 kg, DMS returned **14 579** nondominated solutions.

Madeira et al., *Compos. Part B* 77:391–401, 2015.

Note. The material was encoded by subintervals (aluminium $[1, 2]$, GFRP $[2, 3]$, CFRP $[3, 4]$): this imposes an **artificial order** on a *categorical* variable — a small step can “switch material” just by crossing an arbitrary boundary. Handling categorical variables natively is what DMS-SI-Mix does.

KEY TAKEAWAYS — 6.5

1. DMS is **multiobjective direct search**: instead of one iterate, it keeps a **list** of nondominated solutions.
2. It polls a centre of the list and uses **dominance** to update it; nothing is aggregated.
3. **Success** = the list changed; **failure** reduces the step size of the polled centre.
4. One run can build an **approximation** with many nondominated points, even for nonconvex fronts.
5. The theory guarantees Pareto–Clarke stationarity of a limit point, under assumptions — a local necessary condition, not convergence of the list to the global front.

What if, instead of a list built by polling, we keep a population that evolves? → **6.6 NSGA-II**

References Full list: *Course bibliography*, at the end of the course PDF.

- Custódio, A. L., Madeira, J. F. A., Vaz, A. I. F., Vicente, L. N., *Direct multisearch for multiobjective optimization*, SIAM Journal on Optimization 21(3):1109–1140, 2011 — the DMS paper.
- Custódio, A. L., Madeira, J. F. A., *MultiGLODS: global and local multiobjective optimization using direct search*, Journal of Global Optimization 72(2):323–345, 2018 — MultiGLODS.
- Madeira, J. F. A., *Multiobjective and global optimization with directional direct search*, PhD thesis in Mathematics, IST, Universidade de Lisboa, 2019.

Resources

- Beyond the course: *DMS-SI-Mix*, with scale invariance and continuous, *discrete* (ordered values: number of layers, catalogue thicknesses) and *categorical* (unordered options: the material) variables handled natively.
- Original MATLAB code of DMS, distributed in class: www.mat.uc.pt/dms (not a course function).